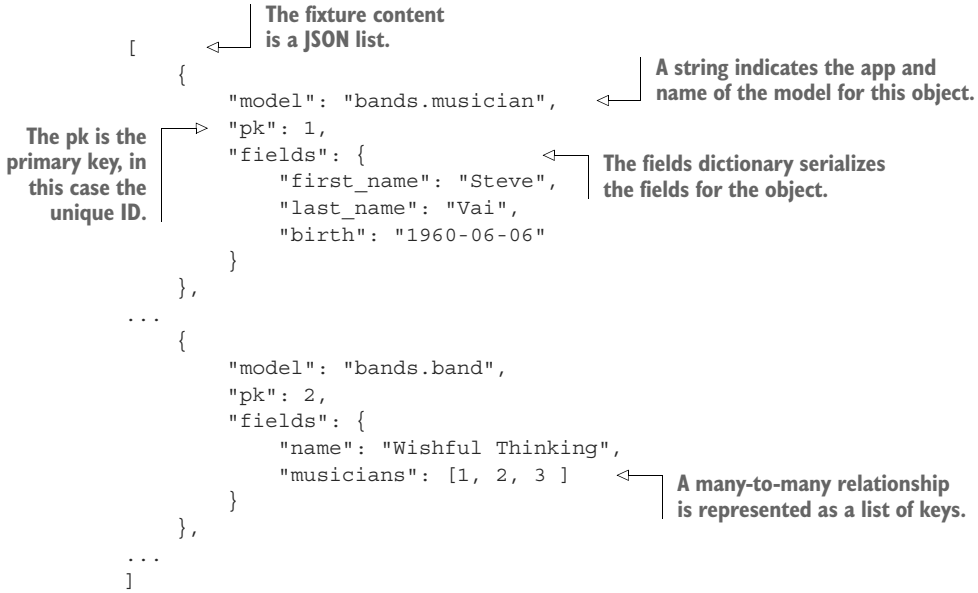
JSON format by default. The command optionally takes a list of apps whose data you are interested in. As the data is in JSON, the `json.tool` module built into Python can be handy; running it directly through `python -m` pretty prints the output. The following line dumps the band app to the screen in JSON, in a human readable format:

```
(venv) RiffMates$ python manage.py dumpdata bands | python -m json.tool
```

The output is a JSON representation of all the musicians, bands, venues, and rooms you created in the database. The content is serialized as a JSON list. The following shows two portions of the output, the serialized Steve Vai `Musician` object and the serialized Wishful Thinking `Band`:




```
[
    {
        "model": "bands.musician",
        "pk": 1,
        "fields": {
            "first_name": "Steve",
            "last_name": "Vai",
            "birth": "1960-06-06"
        }
    },
...
    {
        "model": "bands.band",
        "pk": 2,
        "fields": {
            "name": "Wishful Thinking",
            "musicians": [1, 2, 3 ]
        }
    },
...
]
```

The companion to `dumpdata` is intuitively named `loaddata`. You specify what data to load via a filename or an app name. Dump the data again, this time without the pretty printing, and redirect the content into a file named bands.json:

```
(venv) RiffMates$ python manage.py dumpdata bands > bands.json
```

When your IDE is not your friend

You have to be careful not to accidentally corrupt a fixture output by `dumpdata`, as you might make it impossible to load back in again later. Although a fixture is just a text file, there are text files, and then there are *text files*. Different operating systems use different characters to indicate a newline. Some editors will insert a *byte-order mark* (BOM) (https://www.unicode.org/faq/utf_bom.html#bom1) when encountering Unicode content.